

1 Experiment 2 – Additional Assignment: Perceptron Learning Algorithm for XOR Gate

1.1 Objective

To implement the Perceptron Learning Algorithm for the XOR gate using Python, display the weights after every weight update, plot the decision boundary after each update, and analyze the pattern that prevents the algorithm from converging.

1.2 Theory

A perceptron is one of the simplest forms of an artificial neural network. It consists of input features connected directly to an output neuron. The neuron calculates a weighted sum of the inputs and applies an activation function to produce the output.

For two input features, the net input is calculated as:

$$z = w_1x_1 + w_2x_2 + b \quad (1)$$

where w_1 and w_2 are the weights, x_1 and x_2 are the input features, and b is the bias. A binary step activation function is used:

$$\hat{y} = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases} \quad (2)$$

The perceptron learning rule updates the weights whenever a training sample is incorrectly classified:

$$w_i^{new} = w_i^{old} + \eta(y - \hat{y})x_i \quad (3)$$

The bias is updated using:

$$b^{new} = b^{old} + \eta(y - \hat{y}) \quad (4)$$

where η is the learning rate, y is the target output, and $\hat{y}$ is the predicted output.

The decision boundary of the perceptron is given by:

$$w_1x_1 + w_2x_2 + b = 0 \quad (5)$$

For a two-dimensional input, this equation represents a straight line.

1.3 XOR Gate

The XOR (Exclusive OR) gate produces an output of 1 when the two input values are different and produces an output of 0 when they are the same.

The truth table used in this experiment is shown in Table 1.

Table 1: Truth Table of the XOR Gate

x_1	x_2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

The class-0 samples are:

$$(0, 0), \quad (1, 1) \tag{6}$$

The class-1 samples are:

$$(0, 1), \quad (1, 0) \tag{7}$$

1.4 Perceptron Learning Algorithm

The following procedure was used to train the perceptron:

1. Initialize the weights and bias.
2. Select the learning rate.
3. Calculate the weighted sum for each training sample.
4. Apply the binary step activation function.
5. Calculate the prediction error.
6. Update the weights and bias for incorrectly classified samples.
7. Record the updated weights and bias.
8. Repeat the process for the specified number of epochs.
9. Plot the decision boundary after every weight update.
10. Analyze the weight-update pattern and convergence behavior.

1.5 Implementation

The Perceptron Learning Algorithm was implemented in Python using NumPy. Matplotlib was used to visualize the decision boundary after every weight update.

The learning rate used in the experiment was:

$$\eta = 0.1 \quad (8)$$

The perceptron was trained for a fixed number of epochs to demonstrate its behavior on the non-linearly separable XOR dataset.

1.6 Weight Updates

The weights and bias obtained after every misclassification and subsequent update were recorded.

Table 2: Weights and Bias After Each Perceptron Update

Upd.	Epoch	Sample	x_1	x_2	Target	Pred.	Error	w_1	w_2	b
1	—	—	—	—	—	—	—	—	—	—
2	—	—	—	—	—	—	—	—	—	—
3	—	—	—	—	—	—	—	—	—	—
4	—	—	—	—	—	—	—	—	—	—
5	—	—	—	—	—	—	—	—	—	—
6	—	—	—	—	—	—	—	—	—	—
7	—	—	—	—	—	—	—	—	—	—
8	—	—	—	—	—	—	—	—	—	—

Note: The entries in this table should be replaced with the actual weight-update values obtained from the Python implementation.

1.7 Decision Boundary Visualization

The perceptron decision boundary is defined by:

$$w_1x_1 + w_2x_2 + b = 0 \quad (9)$$

Since the weights are modified after every misclassification, the decision boundary also changes during training.

1.7.1 Decision Boundary Evolution

The complete evolution of the decision boundary after successive weight updates is shown in Figure 1.



Figure 1: Evolution of the perceptron decision boundary after successive weight updates for the XOR dataset.

Inference: The decision boundary continuously changes as the perceptron updates its weights. However, no single straight-line boundary can correctly separate the two XOR classes, causing the boundary to change repeatedly without reaching a stable configuration.

1.7.2 Decision Boundary After Individual Updates

The decision boundary after individual weight updates was also generated to visualize the effect of each update.

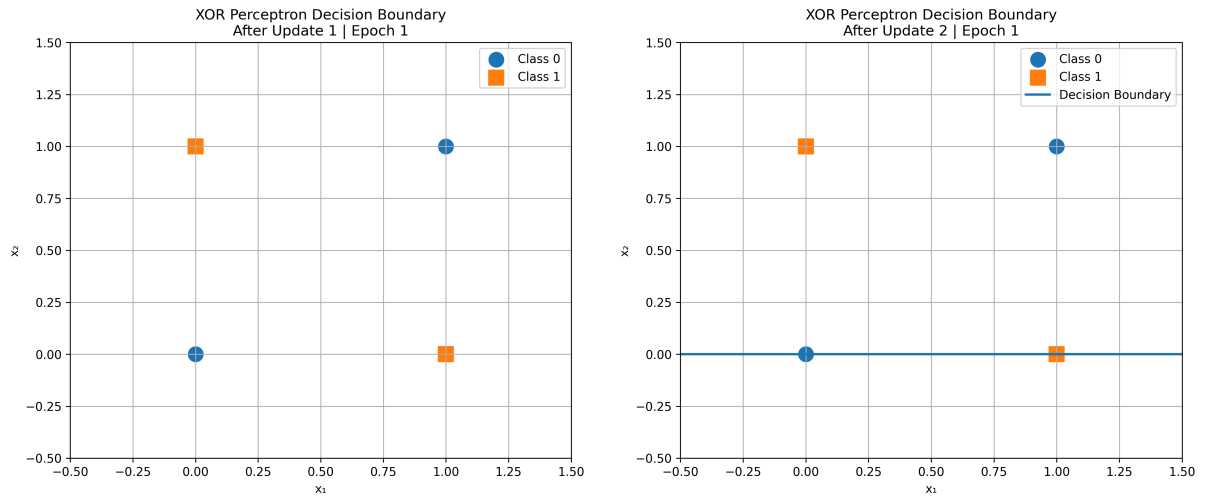


Figure 2: Decision boundaries after the first two perceptron weight updates.

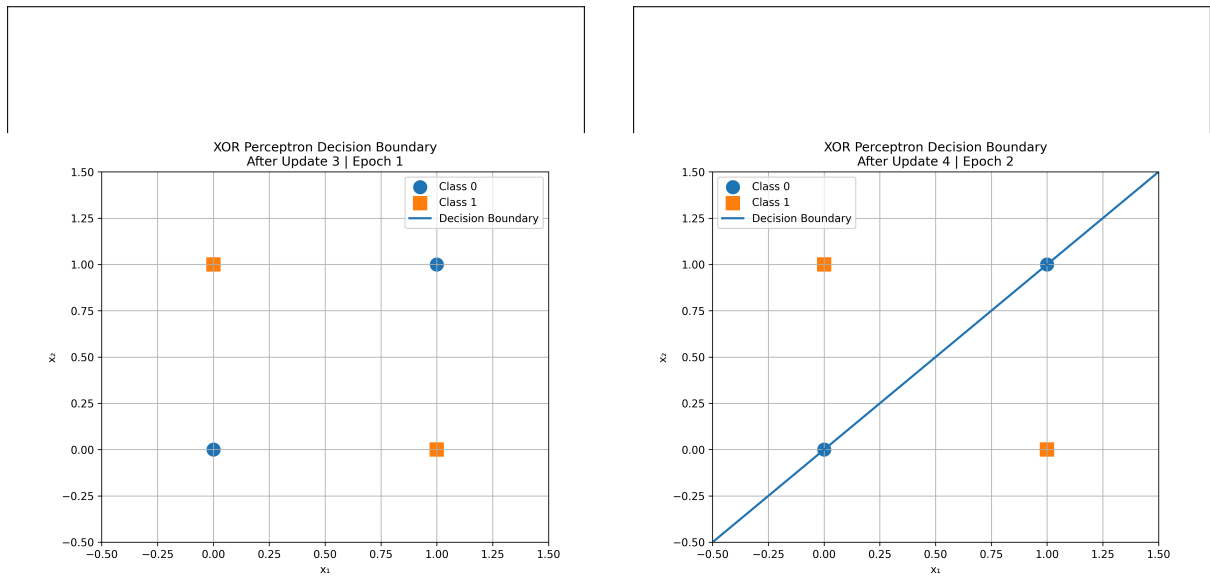
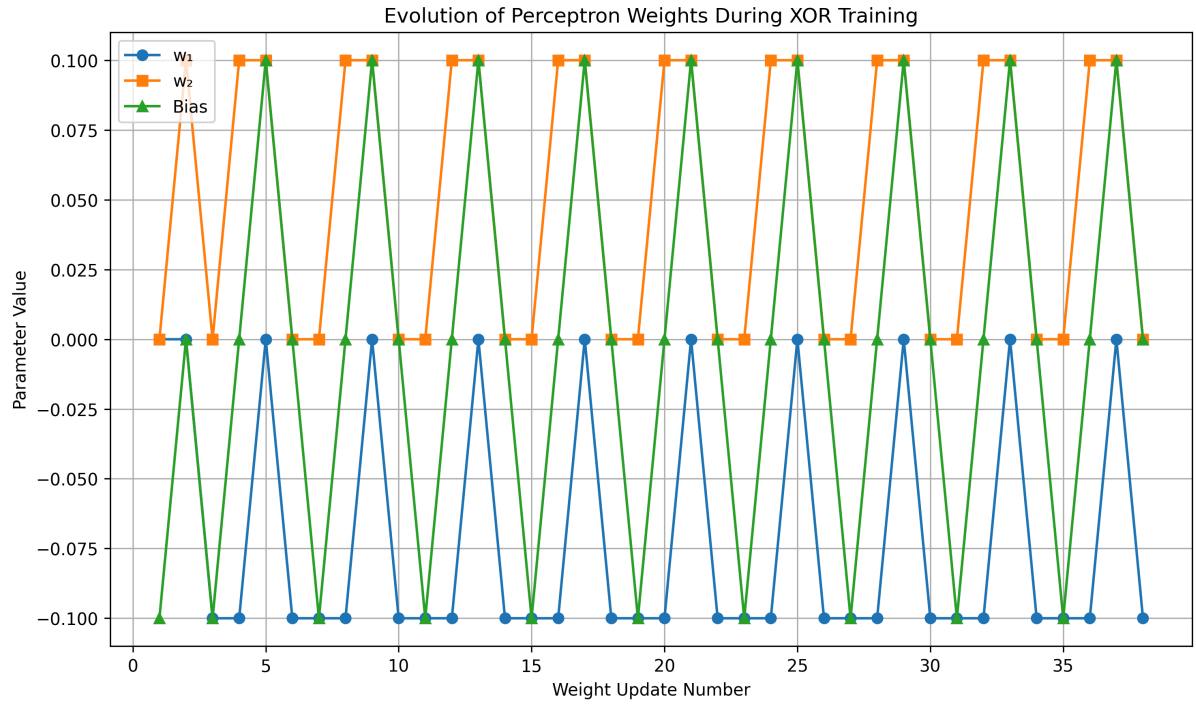


Figure 3: Decision boundaries after subsequent perceptron weight updates.

Inference: The individual plots show that each weight update changes the orientation or position of the decision boundary. The boundary does not settle into a position capable of separating all four XOR samples.

1.8 Weight Evolution

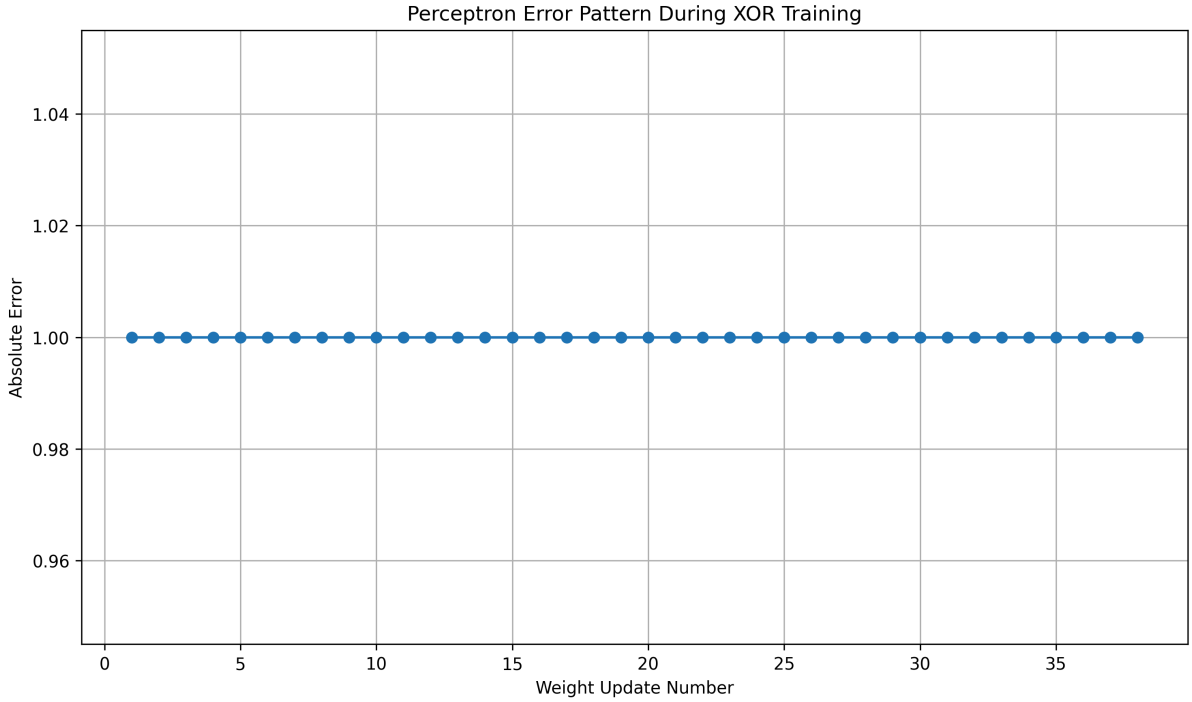
The variation of the weights and bias throughout training was visualized.



Inference: The weights and bias continue to change during training instead of settling at a stable configuration. This indicates that the perceptron repeatedly attempts to correct different misclassified samples.

1.9 Error Pattern

The classification error observed during successive updates was analyzed.



Inference: Errors continue to occur during training because correcting one incorrectly classified XOR sample can result in another sample becoming incorrectly classified. Hence, the perceptron does not reach a zero-error stable solution.

1.10 Analysis of Non-Convergence

The XOR dataset is not linearly separable. The positive samples $(0, 1)$ and $(1, 0)$ lie diagonally opposite to the negative samples $(0, 0)$ and $(1, 1)$.

A single-layer perceptron can only learn a linear decision boundary of the form:

$$w_1x_1 + w_2x_2 + b = 0 \quad (10)$$

Therefore, it is impossible for a single straight line to correctly separate the two classes of the XOR dataset.

1.10.1 Observed Weight-Update Pattern

The following pattern was observed during training:

1. The perceptron encounters a misclassified sample.
2. The weights and bias are updated.
3. The updated decision boundary may correctly classify the current sample.
4. The new boundary may cause another XOR sample to become misclassified.
5. The perceptron updates the weights again.
6. The process continues across successive epochs.

This produces an **oscillatory weight-update pattern** rather than convergence.

1.11 Linear Separability

For a linearly separable dataset, a single hyperplane can divide the input space into two regions corresponding to the two classes.

For XOR, the class labels are:

$$(0, 0) \rightarrow 0 \tag{11}$$

$$(0, 1) \rightarrow 1 \tag{12}$$

$$(1, 0) \rightarrow 1 \tag{13}$$

$$(1, 1) \rightarrow 0 \tag{14}$$

The alternating arrangement of the classes means that no single linear decision boundary can correctly classify all four points.

Therefore, the single-layer perceptron is unable to converge to a perfect solution for XOR.

1.12 Result

The Perceptron Learning Algorithm was successfully implemented for the XOR gate using Python.

The following observations were obtained:

- The weights were updated after every misclassified training sample.
- The weights and bias after each update were recorded.
- The decision boundary was plotted after successive weight updates.
- The weights did not converge to a stable solution.
- The perceptron failed to correctly classify all four XOR input combinations.
- The observed oscillatory update pattern was caused by the non-linearly separable nature of XOR.

1.13 Conclusion

The Perceptron Learning Algorithm was successfully implemented and analyzed for the XOR gate. The experiment demonstrated the weight-update mechanism and the corresponding movement of the decision boundary.

The single-layer perceptron failed to converge because XOR is not linearly separable. A single-layer perceptron can construct only a linear decision boundary and therefore cannot correctly classify the XOR dataset.

The continuous modification of the weights and decision boundary demonstrates the oscillatory behavior responsible for non-convergence. This experiment highlights the fundamental limitation of a single-layer perceptron and motivates the use of Multi-Layer Perceptrons with hidden layers and nonlinear activation functions for solving nonlinear classification problems.